

Lecture 22:
Deep Learning
Advancement in CNN and RNN
ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- **Recall:** Innovations in Convolutional Neural Networks
- More advancement
 - From classification to localization
 - From classification to semantic segmentation
- RNN

From Classification to Localization

- How CNNs learned to not only see but also localize.
- Two-stage detectors: R-CNN, Fast R-CNN, Faster R-CNN
- One-stage detectors: YOLO, SSD
- Transformer-based: DETR

R-CNN (2014): Regions with CNN

Motivation

- CNNs (e.g. AlexNet) worked for classification: one object.
- How to extend CNNs to find *many objects*?

Mechanism

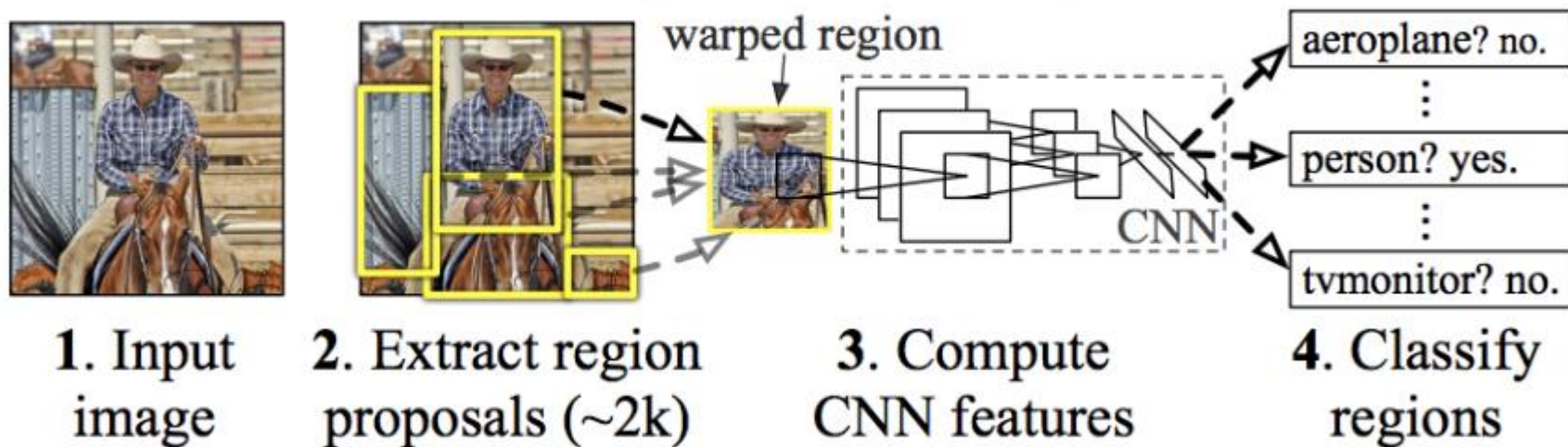
- 1.**Selective Search:** Propose ~2000 candidate regions (ROIs).
- 2.**CNN Feature Extraction:** Run each region through pretrained CNN (e.g., AlexNet).
- 3.**SVM Classifier:** Classify each region.
- 4.**Bounding Box Regression:** Fine-tune box coordinates.

R-CNN

- Before R-CNN, object detection relied on hand-crafted features (HOG, SIFT) and shallow classifiers.
R-CNN was the first to combine deep CNN feature extraction with region-based detection.
- R-CNN is for multiple-object detection, but it does so indirectly by detecting objects one region at a time.
- It does not look at the entire image in one pass (similar to YOLO does later). Instead, it looks at many *proposed regions* and classifies each one as a specific object or background.

R-CNN

R-CNN: *Regions with CNN features*



Fast R-CNN (2015): Shared Features

- **Motivation**

- R-CNN wastes computation: each region runs full CNN.
- Need to reuse features for all regions.

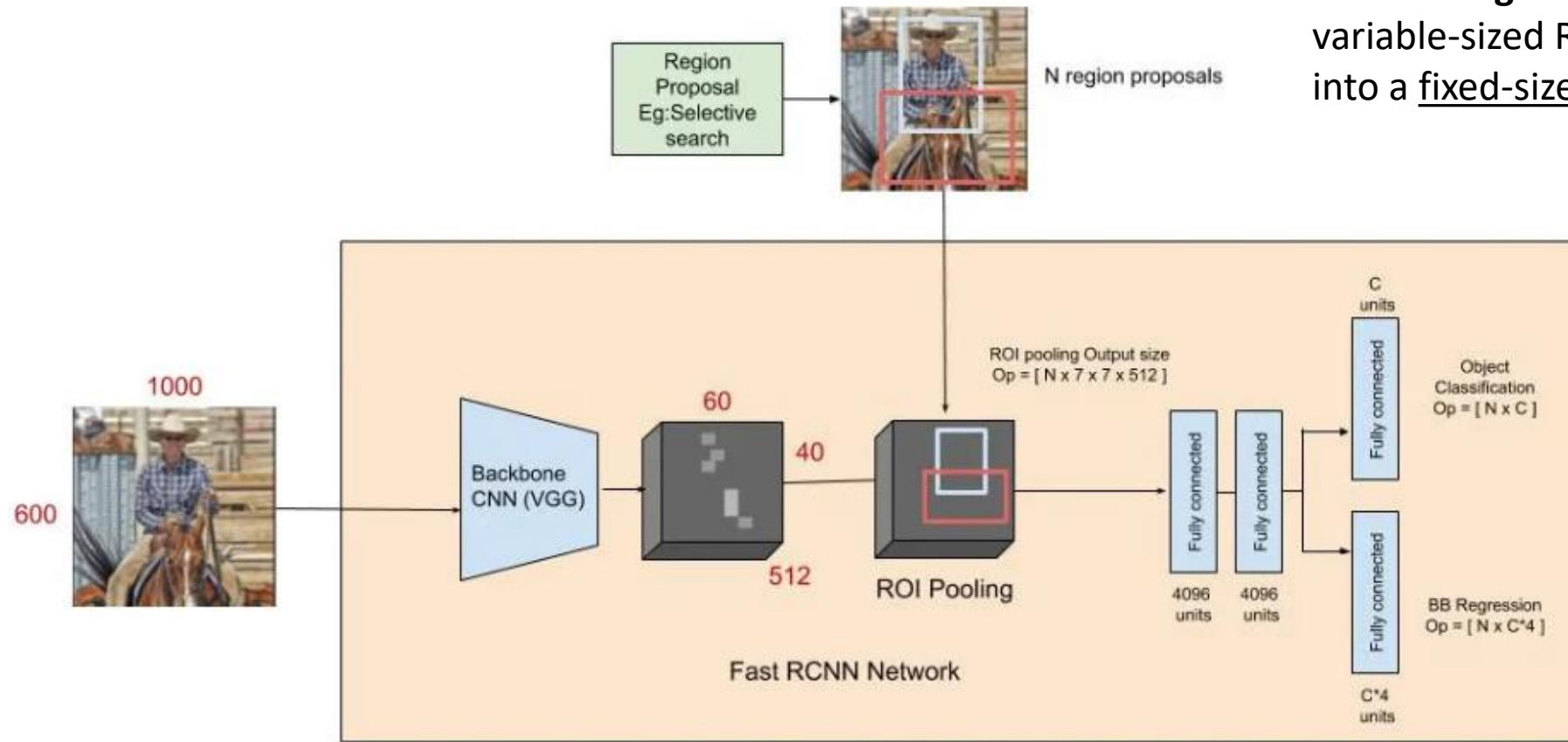
- **Mechanism**

- One CNN over the entire image: shared feature map.
- Each region proposal projected onto this feature map via ROI Pooling.
- Fully connected layers classify and regress box offsets.

Core Mechanism of Fast R-CNN

- **Step 1: Shared Convolutional Feature Map**
 - The entire image is fed once into the CNN backbone (e.g., VGG16).
 - Output: a high-level feature map F of size $H' \times W' \times C$.
 - Each pixel in F corresponds to a receptive field region in the original image.
- **Step 2: Region Proposals**
 - The same region proposals (from Selective Search) are reused: about 2000 boxes.
 - But now, instead of cropping the image, each proposal is mapped to its corresponding region on the feature map F .

Fast R-CNN



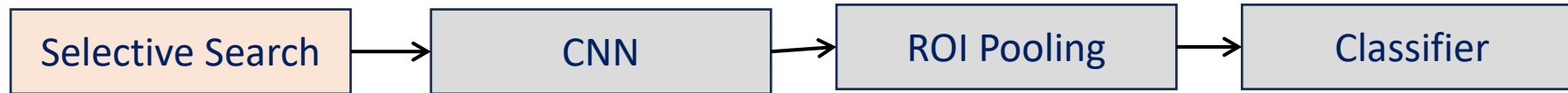
ROI Pooling converts each variable-sized ROI feature patch into a fixed-size feature.

Faster R-CNN: Learning to Propose Regions

- **Why Go Beyond Fast R-CNN?**
- **Fast R-CNN** fixed redundant CNN passes using ROI Pooling, but it still relied on Selective Search for generating ~2000 region proposals.
- **Selective Search Limitations:**
 - Hand-crafted, non-learnable
 - Produces many redundant regions
 - Not adjustable: cannot improve with training data

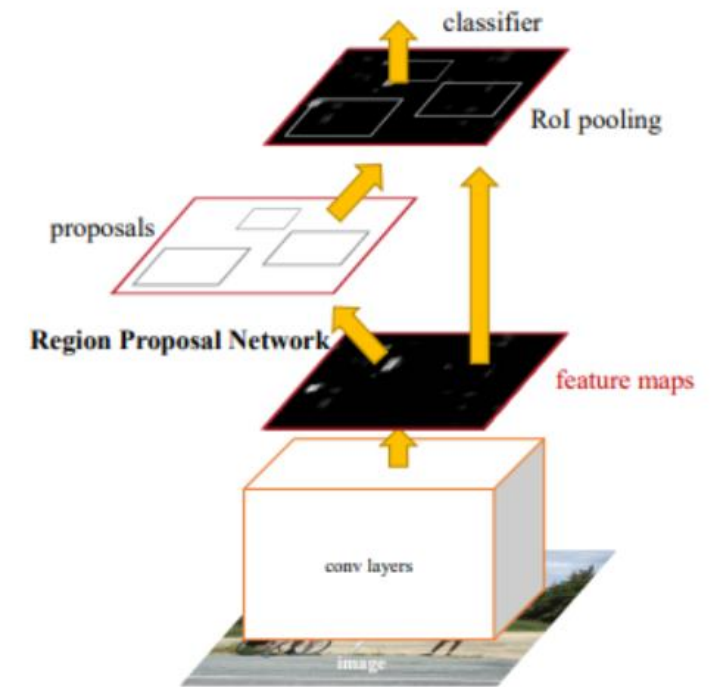
Faster R-CNN

- Introduce a small neural network called a **Region Proposal Network (RPN)** that *learns* to propose candidate object regions directly from CNN feature maps.



Faster R-CNN: Overall Architecture

1. Input Image to Shared CNN Backbone (e.g., VGG16 or ResNet-50)
2. Feature Map to RPN
3. RPN outputs:
 - objectness score (is this region likely to contain an object?)
 - bounding box coordinates
4. The top N proposals (e.g., 300) are sent to ROI Pooling
5. ROI features to classification + refined bounding box regression head



Faster R-CNN architecture

From Two-Stage to One-Stage Detectors

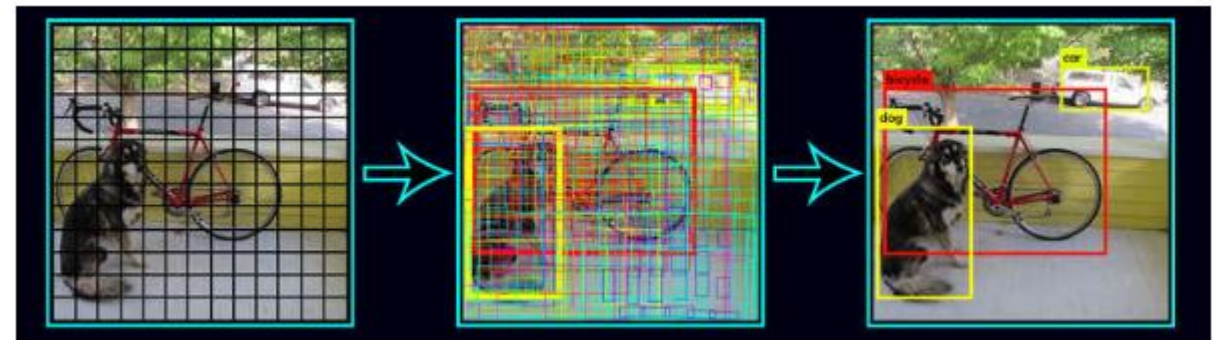
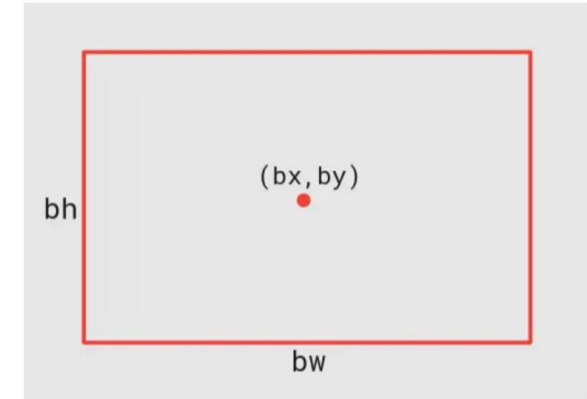
- Two-stage detectors (Faster R-CNN) are accurate but slow.
 - Need region proposals and ROI Pooling and classifier.
- Many real applications (autonomous vehicles, robotics, manufacturing inspection) need *fast inference*.
- **Objective:**
Design a **single-stage** detector that predicts *bounding boxes + classes directly from one CNN pass*.

YOLO: You Only Look Once (2016)

Turn detection into a single regression problem from (image pixels) to (bounding boxes and class probabilities).

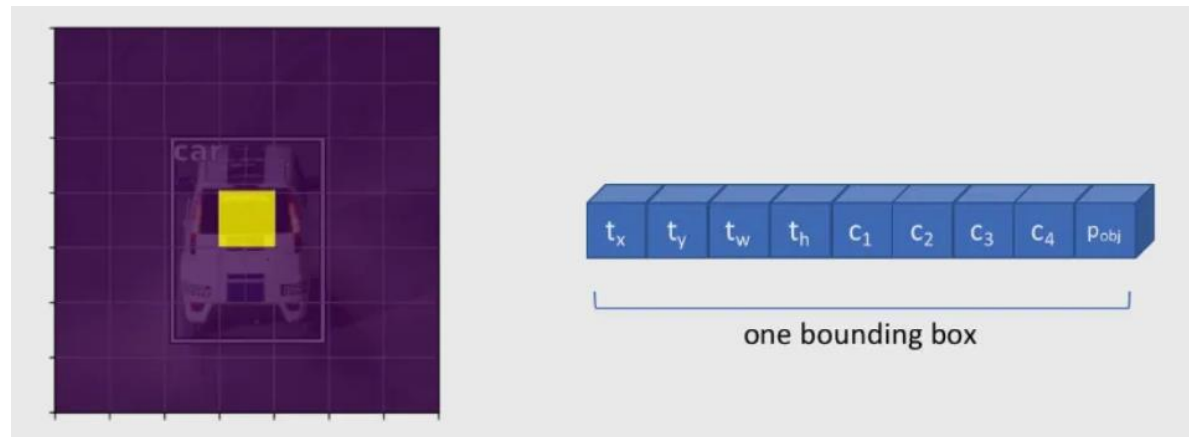
Mechanism:

1. Resize image: feed once through CNN.
2. Divide image into an $S \times S$ grid (e.g., 7×7).
3. Each grid cell predicts:
 1. B boxes (x, y, w, h , confidence)
 2. C class probabilities.



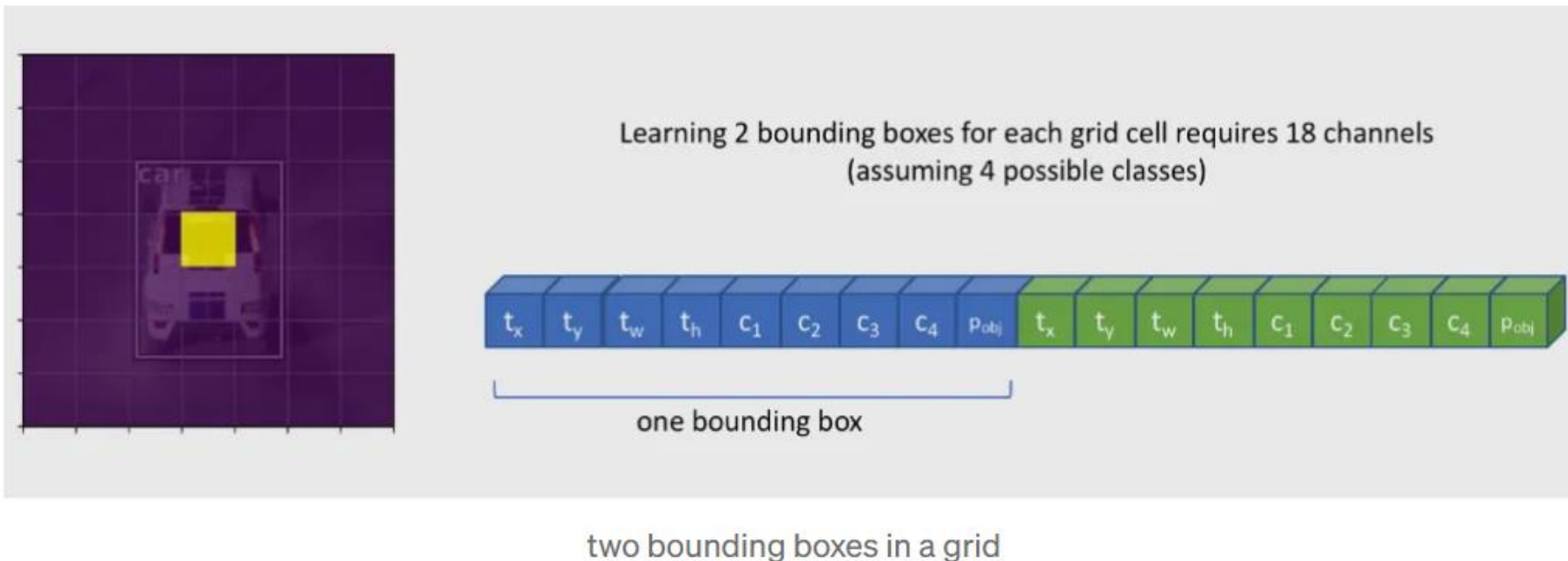
Information predicted for each grid cell in the feature map

- The prediction for each grid cell combines several elements:
 1. The likelihood that the cell contains an object, known as the confidence score or objectness.
 2. The class probabilities, indicating which class the object belongs to.
 3. The coordinates and dimensions of the bounding box that localizes the object.



B bounding box for each cell

- For each grid cell, if we predict B bounding boxes, each bounding box prediction includes five components



Note: the two bounding boxes ($B=2$) predicted by each cell in YOLO do not necessarily correspond to two different objects. They are two different guesses or hypotheses for what the object *might* look like, usually differing in shape (aspect ratio) or size.

How YOLO Works

Input: Resize image to 448×448

- **Backbone CNN:** Extract features (Conv layers)
- **Prediction Layer:** Fully connected layer outputs (x, y, w, h, conf, classes) for each cell.
- **Filtering:**
 - Keep boxes with high confidence (e.g., > 0.5)
 - Apply Non-Maximum Suppression (NMS) to remove overlapping boxes

Output:

A set of labeled boxes: *final detected objects*.

YOLO Loss Function

- YOLO learns to optimize localization and classification and confidence jointly.

$$L = L_{coord} + L_{conf} + L_{class}$$

$$L_{coord} = \lambda_{coord} \sum (x - \hat{x})^2 + (y - \hat{y})^2 + (\sqrt{w} - \sqrt{\hat{w}})^2 + (\sqrt{h} - \sqrt{\hat{h}})^2$$

$$L_{conf} = \sum (C - \hat{C})^2, \quad L_{class} = \sum (p(c) - \hat{p}(c))^2$$

YOLO Loss Function

- YOLO divides the image into an $S \times S$ grid (e.g., $7 \times 7 = 49$ cells).
- For each cell:
 - It predicts **B** bounding boxes
each box has $(x, y, w, h, confidence)$
 - It also predicts **C** class probabilities
- Then, YOLO's total loss is **the sum over all cells**:

$$L = \sum_{i=1}^{S^2} L_{cell_i}$$

From Classification to Segmentation

- **Why Move Beyond Classification Networks?**
- Models such as ResNet, DenseNet, and ViT output a single label per image.
- But many real-world problems require dense, spatial predictions:
 - Semantic segmentation: classify every pixel
 - Object detection: localize multiple objects
 - Depth estimation: continuous map
 - Medical imaging: outline organs, cells

We need architectures that can predict for every pixel, not just the whole image.

Revisiting CNN Output Structure

- Why Standard CNNs Cannot Do Segmentation Directly
- **Standard CNN pipeline:**
 - Pooling and FC layers destroy spatial resolution.
 - The network no longer knows *where* features came from.
 - Works for classification, but fails for pixel-wise prediction.
- **Goal:**
Keep convolutional feature extraction,
remove the FC bottleneck,
and recover spatial correspondence.

The Concept of Dense Prediction

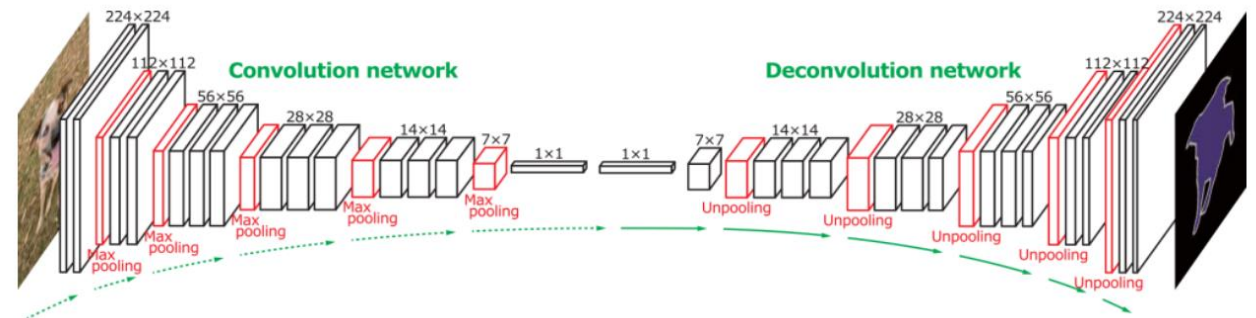
Dense vs. Global Prediction

Task Type	Example	Output	Network Goal
Global Prediction	Image classification	1 label	Is there a cat?
Dense Prediction	Semantic segmentation	Label per pixel	Where is the cat?

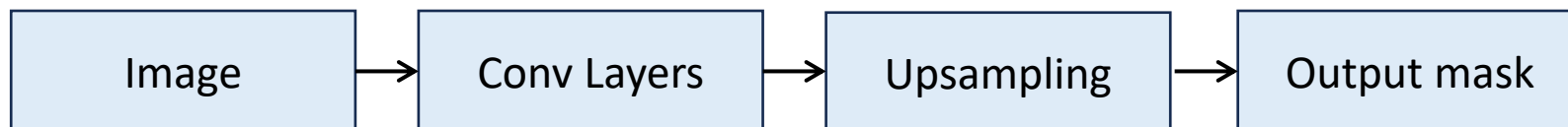
Instead of producing one number (or class),
the model must output a grid of values: a 2-D map aligned with the input image.

Fully Convolutional Networks (FCNs)

- Replace fully connected layers with **convolutions**.
- Output is not a single class but a **pixel-level map**.
- **Applications:**
 - Semantic segmentation
 - Depth estimation
 - Object boundary detection

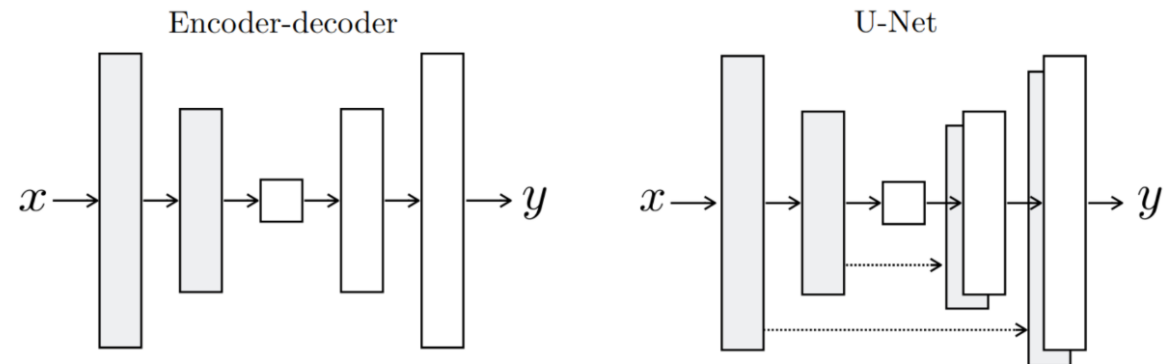


<https://tjmachinelearning.com/lectures/1718/fcn/>



U-Net: Encoder-Decoder for Segmentation

- **Architecture:**
 - **Encoder:** Series of conv and pool to compress information (context)
 - **Decoder:** Upsampling and skip connections to recover spatial detail
 - **Skip connections:** combine low-level (edges) and high-level (context) features
- **Applications:**
- Biomedical image segmentation (e.g., cell, tumor, or organ boundaries)



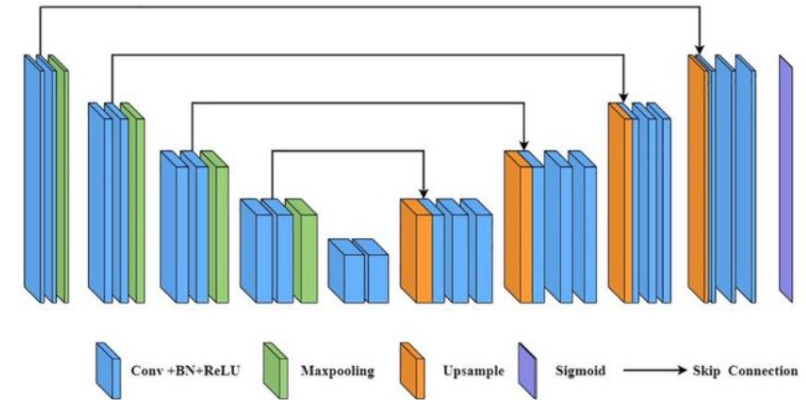
U-Net Details

Stage	Function	Example Output Size
Encoder	Downsampling with Conv and Pool	512×512 to 64×64
Bottleneck	Deepest layer with dense features	64×64
Decoder	Upsampling and concat skip features	Back to 512×512

How Upsampling Works

- Example:
- Let's say you have a 2×2 feature map:
$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$
- After 2× nearest-neighbor upsampling it becomes 4×4:
$$\begin{bmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 3 & 3 & 4 & 4 \\ 3 & 3 & 4 & 4 \end{bmatrix}$$
- This doubles the resolution (each pixel duplicated).

Upsampling in U-Net



Dong, Zhangyu, et al. "L-unet: A landslide extraction model using multi-scale feature fusion and attention mechanism." *Remote Sensing* 14.11 (2022): 2552.

- Each step of the decoder does:
 - **Upsample** the previous low-resolution feature map (using transpose convolution or interpolation and conv).
 - **Concatenate** it with the **skip connection** from the encoder (same spatial size).
 - **Convolve** to fuse details and reduce artifacts.
- This sequence helps the network **progressively reconstruct fine-grained spatial structure**.

Comparison of CNN Variants

Model Type	Input	Output	Common Use
1D CNN	Sequence (T)	Class	Time-series analysis
2D CNN	Image (H×W)	Class	Classification
3D CNN	Video (T×H×W)	Class	Spatio-temporal
Dilated CNN	Image	Dense map*	Segmentation
FCN / U-Net	Image	Pixel mask	Biomedical / scene segmentation

*a dense map can contain *continuous* values,
while a pixel mask usually contains *discrete class labels*.

Recurrent Neural Networks (RNNs)

- RNNs: Learning from Sequential Data
- Applications in Time-Series, NLP, and Sequential Decision-Making

Why Do We Need RNNs?

- Standard neural nets assume **independent inputs**.
- Sequential data has **temporal dependency**.
 - Example: word meaning depends on previous words.
 - Example: sensor value today depends on past readings.

RNN Structure

- **The Main Idea: Recurrent Connection**

Equation:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

- x_t : input at time step t
- h_t : hidden state (memory)
- W_h, W_x : weight matrices
- Recurrent link carries *context* forward in time.

Unrolling the RNN

- Show the RNN unrolled through time steps:

$$h_t = f(W_h h_{t-1} + W_x x_t)$$

- **Main Idea:**
- Shared weights between time steps.
- Facilitates handling sequences of variable length.

Applications of RNNs

Domain	Example Task	Description
NLP	Text Generation	AI writes essays
Speech	Speech-to-text	Sequence of audio to text
Finance	Stock Prediction	Predict next-day price
IoT / Manufacturing	Sensor Fault Detection	Detect anomalies in time-series data

The Vanishing Gradient Problem

- **Problem:** When backpropagating through many time steps, gradients become very small.
- **Therefore:**
 - Long-term dependencies are lost.
 - Training becomes unstable.

The Vanishing Gradient Problem

- **Problem:** When backpropagating through many time steps, gradients become very small.
- **Therefore:**
 - Long-term dependencies are lost.
 - Training becomes unstable.

How to Handle Long Dependencies?

Long Short-Term Memory (LSTM)

- **Solution:** Add a *memory cell* with gated control.

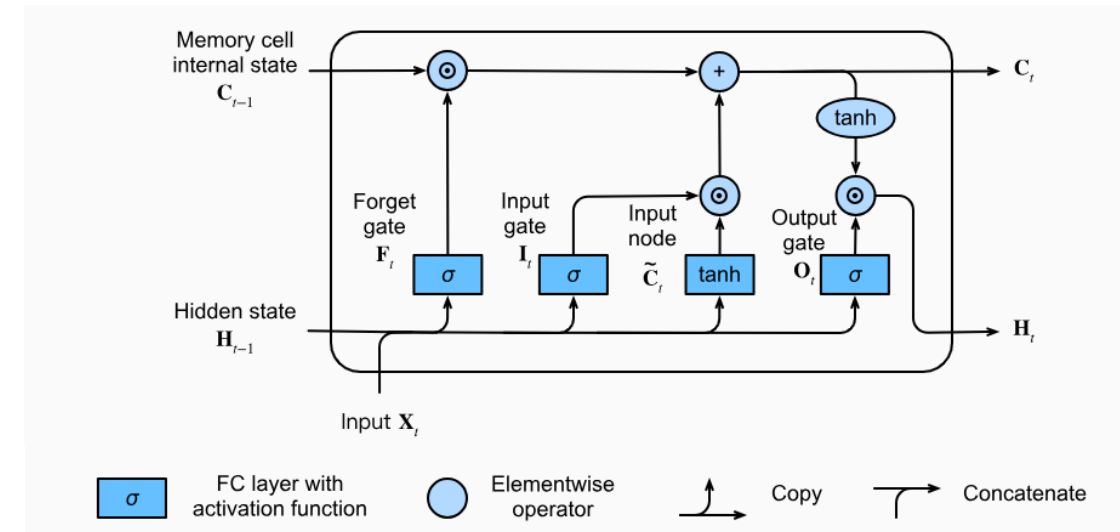
- **Major Components:**

- Cell state C_t : long-term memory.
- Hidden state h_t : short-term output.
- Gates: input i_t , forget f_t , output o_t .

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

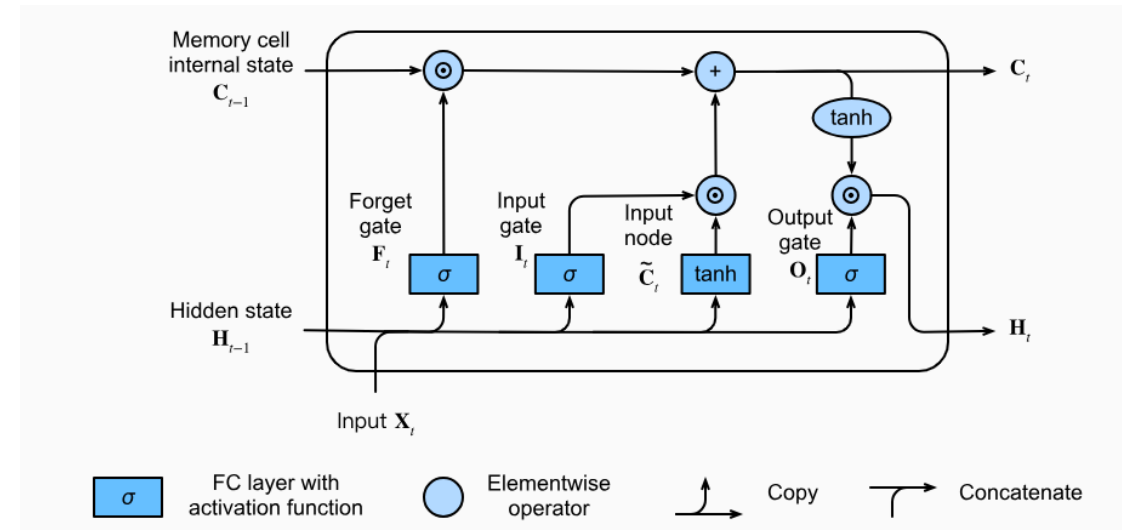
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad h_t = o_t * \tanh(C_t)$$



Intuitive Walk-Through

At each step:

1. **Forget** irrelevant past info.
2. **Input** relevant new data.
3. **Update** cell memory C_t .
4. **Output** filtered version as h_t .



The Main LSTM equations:

Symbol	Name	Role
(C_t)	Cell State	Long-term memory (what the network remembers over many time steps)
(h_t)	Hidden State	Short-term memory or output (what the network outputs now and passes to the next layer)

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$h_t = o_t * \tanh(C_t)$$

Summary of LSTM

Step	Equation	Gate Type	Simple Meaning
1	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	Forget	What old info to erase
2	$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	Input	What new info to add
3	$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$	Candidate	New possible info
4	$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$	Update	Update long-term memory
5	$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	Output	What to reveal as output
6	$h_t = o_t * \tanh(C_t)$	Output Result	Final output and short-term memory

A neural memory system that decides what to remember, what to forget, and what to share at each time step.

Code Example (PyTorch)

```
import torch.nn as nn

rnn = nn.LSTM(input_size=10, hidden_size=20, num_layers=2, batch_first=True)
output, (hn, cn) = rnn(x)  # x: (batch, seq_len, input_size)
```

- When using `batch_first=True`, PyTorch expects input shaped as:
(*batch size, sequence length, input size*)

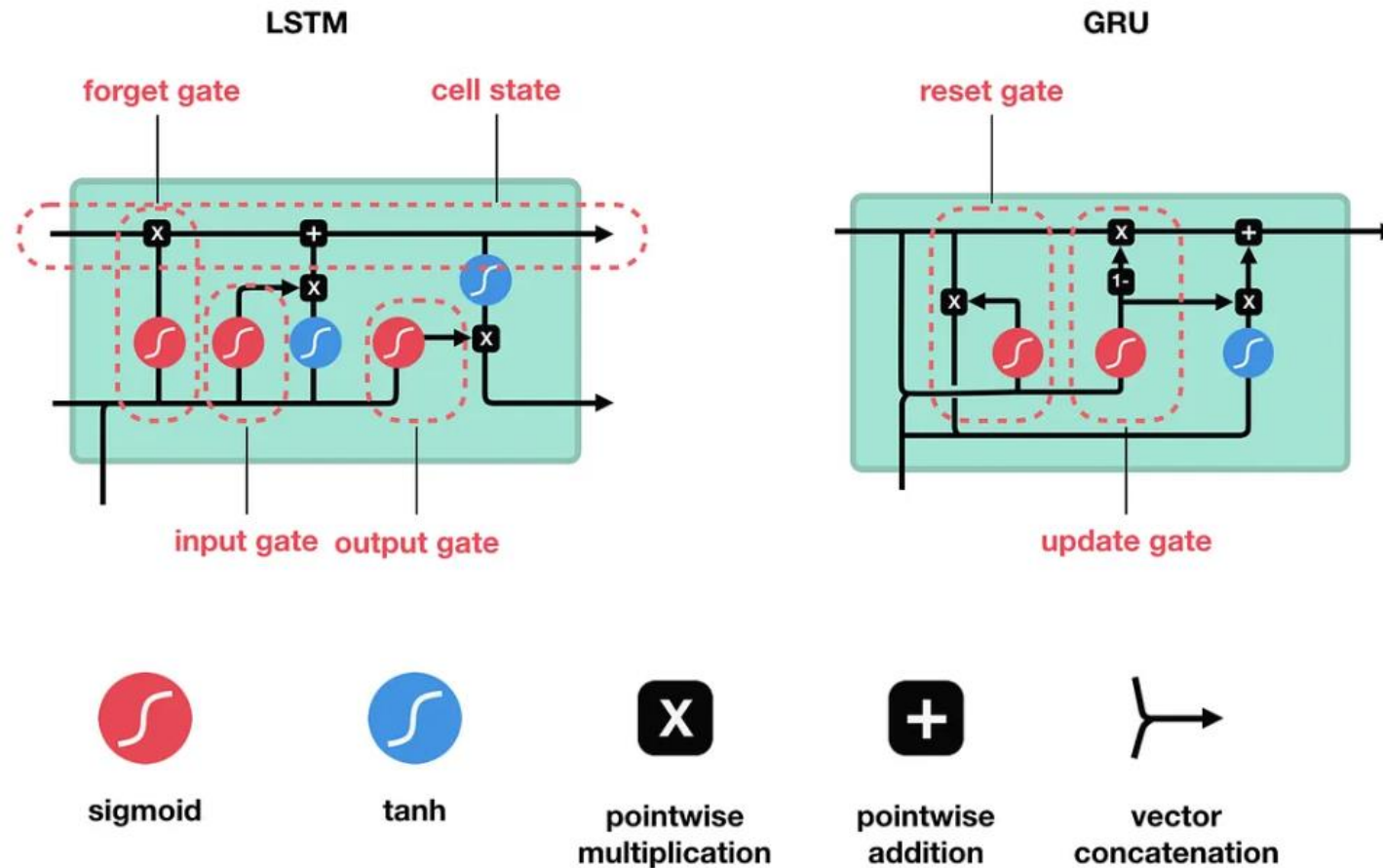
```
x = torch.randn(32, 50, 10)
```

- Batch size: 32, Number of sequences processed in parallel
- Sequence length: 50, Number of time steps per sequence
- Input size: 10, Number of features per time step

Gated Recurrent Unit (GRU): A Simpler Way to Remember

- Developed by Cho et al. (2014) as a more efficient variant of the LSTM.
- LSTM is powerful but computationally heavy (3 gates, cell state, and more parameters).
- Researchers sought a simpler architecture with:
 - fewer parameters
 - faster training
 - similar ability to capture long-term dependencies.

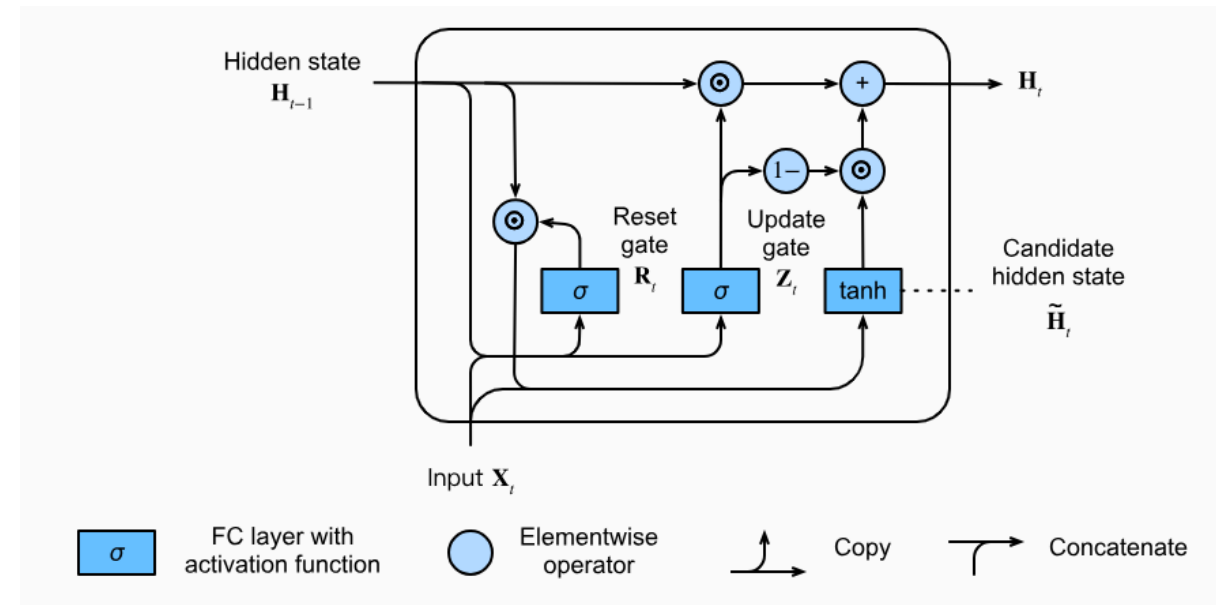
GRU vs. LSTM



Gated Recurrent Unit (GRU)

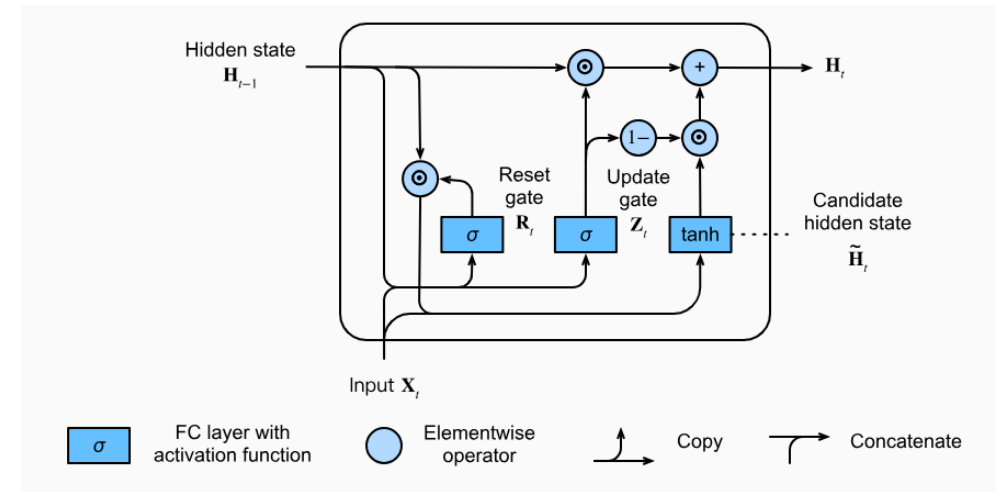
- **Simpler alternative to LSTM.**
- GRU = LSTM with merged gates and no separate cell state.
- Combine the *forget* and *input* behavior into one gate: the **update gate**.
- **Gates:**
- Update gate z_t
- Reset gate r_t
- One hidden state h_t (no separate Ct).
- **Benefits:**
 - Fewer parameters
 - Comparable performance to LSTM
 - Faster training

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

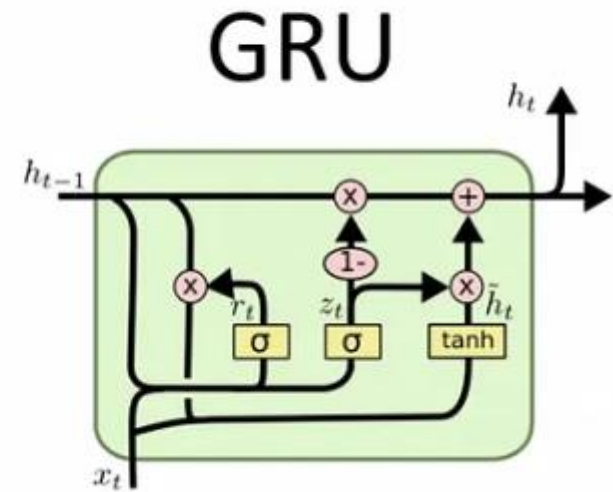
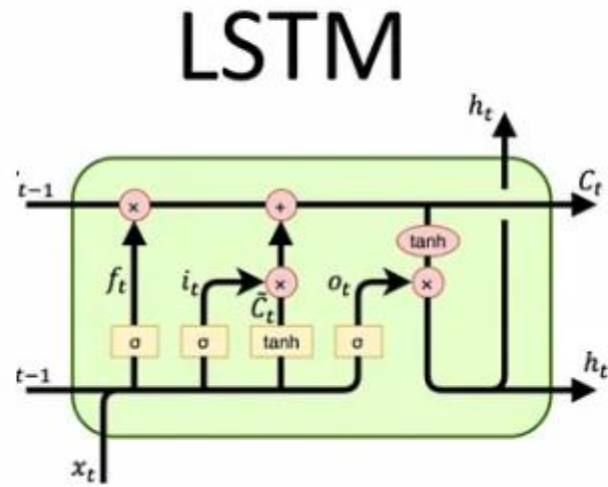
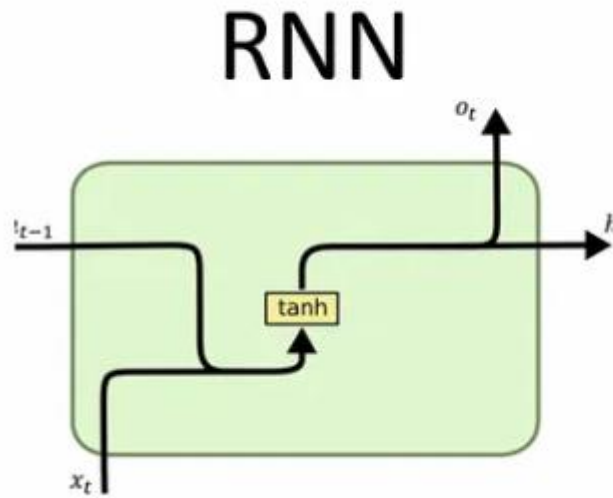


GRU Equations

- GRU combines memory and output into one state.



Gate	Equation	Meaning
Update gate	$z_t = \sigma(W_z[x_t, h_{t-1}])$	How much of the past to keep
Reset gate	$r_t = \sigma(W_r[x_t, h_{t-1}])$	How much of the past to forget
Candidate activation	$\tilde{h}_t = \tanh(W_h[x_t, (r_t * h_{t-1})])$	New memory
Final output	$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$	Blend old and new info



Limitations of RNNs

Limitation	Description
Sequential processing	Must process time steps one by one, slow for long sequences
Long-term memory	Even LSTMs struggle with very long dependencies
Parallelization	Hard to train efficiently on GPUs
Fixed context window	Difficult to connect distant words or tokens

Transformers solve all of these with *attention*

Transformers: The Power of Attention

- Instead of remembering the past step-by-step, the Transformer looks at all positions at once and decides what to pay attention to.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- **Parallel Processing:** Because attention uses matrix multiplications, all tokens are processed in parallel.
 - This eliminates the sequential dependency of RNNs and facilitates massive GPU acceleration.
 - Each token *looks at* all other tokens and decides *how much to focus* on each one.

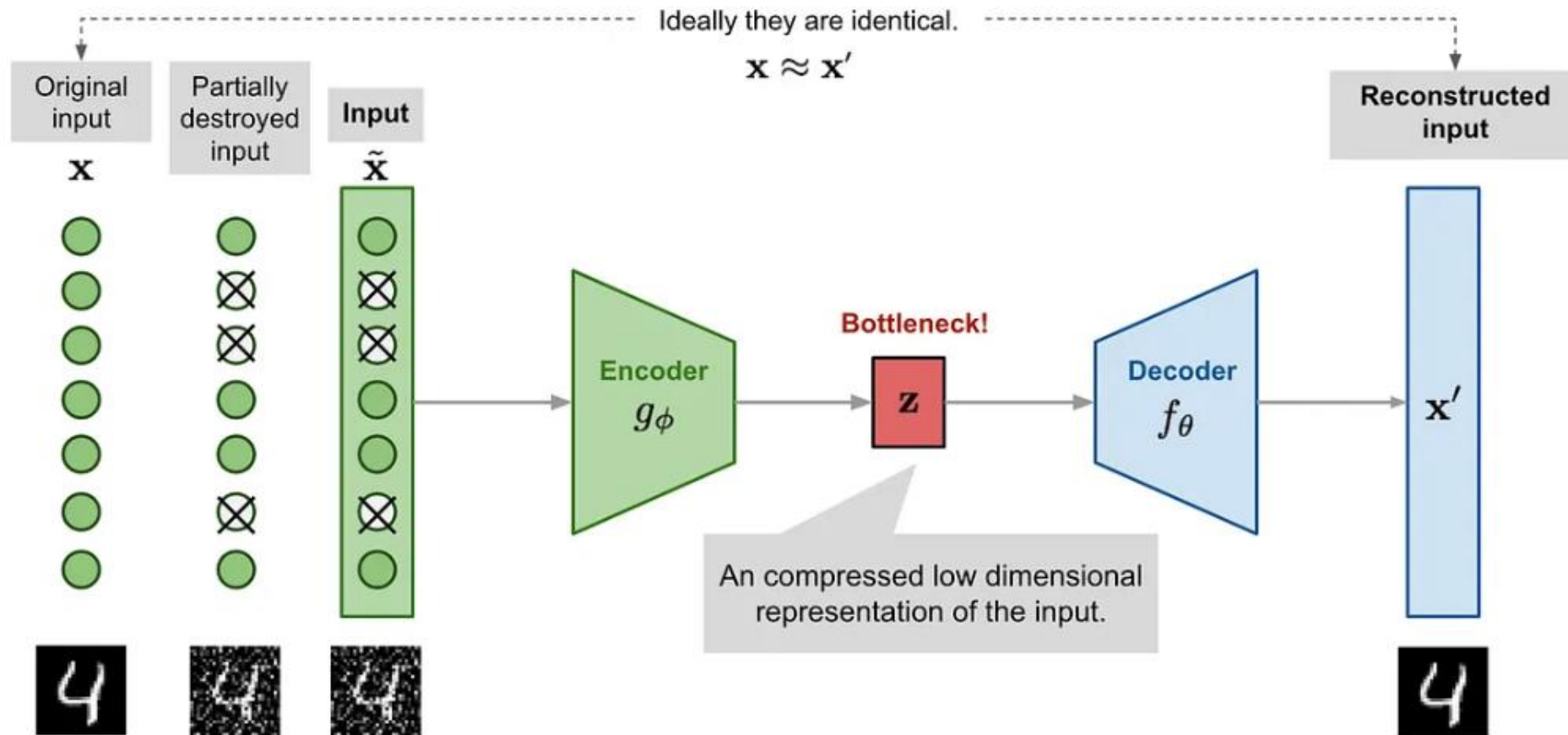
Unsupervised Learning and Representation Learning

- Neural networks learn to compress and reconstruct data.

Autoencoders: Neural Networks for Unsupervised Representation Learning

- We have seen sequence models that predict next words. Now let's look at networks that reconstruct their own input.
- **Why Autoencoders?**
 - Dimensionality Reduction: Learn compact, lower-dimensional representations
 - Feature Learning: Capture meaningful structure without labels
 - Denoising: Reconstruct clean data from noisy input
 - Anomaly Detection: Identify outliers via reconstruction error

Example: Image Denoising Using Autoencoders



Basic Architecture

- Analogy: A smart photocopier that learns to recreate input documents, keeping only the essential details.
- Encoder: compresses input x to latent vector z
- Decoder: reconstructs x' from z
- Training Objective: make x' as close to x as possible

$$z = f_{\theta}(x) \quad (\text{Encoder})$$

$$x' = g_{\phi}(z) \quad (\text{Decoder})$$

$$L = \|x - x'\|^2 \quad (\text{Reconstruction loss})$$

Variational Autoencoders (VAE): Probabilistic Representation Learning

- We saw how classical autoencoders learn to reconstruct data. But what if we want them to generate new, realistic samples, not just copy inputs?
- Instead of one latent code per input, the network learns a distribution.
- Adds probabilistic latent variables:

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

- Loss:

$$L = \|x - x'\|^2 + D_{KL}(q(z|x) \| p(z))$$

Variational Autoencoders (VAE): Probabilistic Representation Learning

- We saw how classical autoencoders learn to reconstruct data. But what if we want them to generate new, realistic samples, not just copy inputs?
- Instead of one latent code per input, the network learns a distribution.
- In a standard autoencoder:

$$x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} \hat{x}$$

- z is a fixed vector per input.
- In a VAE:

$$x \xrightarrow{\text{Encoder}} (\mu, \sigma) \xrightarrow{\text{Sampling}} z \xrightarrow{\text{Decoder}} \hat{x}$$

The encoder outputs a distribution (mean and variance) over possible latent representations.

Kullback-Leibler Divergence

- In a Variational Autoencoder:
 - $q_{\phi}(z | x)$ the encoder's learned distribution over latent variables (posterior).
 - $p(z)$ the prior distribution, usually $\mathcal{N}(0, I)$.
- The **KL divergence term**: $D_{KL} \left(q_{\phi}(z | x) \parallel p(z) \right)$
- encourages the learned latent distribution $q(z | x)$ to stay close to the standard normal $\mathcal{N}(0, I)$.
- **Recall**: DKL measures how different one probability distribution P is from another distribution Q .

$$D_{KL}(P \parallel Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$$

Kullback-Leibler Divergence

- The **KL divergence term**: $D_{KL} \left(q_{\phi}(z | x) \parallel p(z) \right)$
 - Without this term: The encoder might learn arbitrary, scattered distributions for each input.
 - With KL divergence: Latent representations are pushed toward a shared Gaussian structure.

VAE

- VAE is used to learn a compressed representation of data while also being able to generate new, synthetic data points.
- VAEs incorporate probabilistic inference, helping them generate new samples by sampling from a learned latent distribution.

